

Hezbollah forces crossed the Israel–Lebanon border and attacked Israeli military positions while firing rockets and mortars at Israeli towns, sparking a five-week war. https://en.wikipedia.org/wiki/2006_Hezbollah_cross-border_raid

Wiktionary's word of the day:

fortuitous: 1. Happening by fortune or chance; accidental, coincidental. 2. Having good fortune or success; fortunate, lucky. 3. (law) Happening independently of human control; uncontrollable. 4. About Word of the Day 5. Nominate a word 6. Leave feedback <https://en.wiktionary.org/wiki/fortuitous>

Wikiquote quote of the day:

The best preparation for tomorrow is to do today's work superbly well. --William Osler https://en.wikiquote.org/wiki/William_Osler

FRED TALKS

12th July 2026

CONTENTS

Agentic search - retrieval, harness, or model?

DOUG TURNBULL · 1535 WORDS

LLM Judges aren't the shortcut you think

DOUG TURNBULL · 1869 WORDS

A non-anthropomorphized view of LLMs

HALVAR.FLAKE · 1478 WORDS

[Daily article] July 12: Wikipedia:Featured topics/Liverpool F.C.

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 337 WORDS

Agentic search - retrieval, harness, or model?

DOUG TURNBULL · 09 JUN 2026 · [SOURCE](#)

Agentic search gets interesting when agents do not know how to find the right answer.

Oh, the agent might think it knows. It might confidently BS us. But the agent’s poor domain intuition steers itself astray.

Agents make false assumptions about what *our* users think is relevant. Our fashionista users think “red shoes” should return high-heels. When I worked at one company ABE wasn’t a president, it was an A/B testing tool. Agents need context to know these things - and [context engineering needs agentic search](#)

Confusingly, depending who you talk to, agentic search can mean one of three distinct implementation patterns:

- **Retrieval-centric** - just build good search so agents can use it to fill in missing context
- **Harness-centric** - steer the agent towards needed context, even with bad search
- **Model-centric** - fine tune an LLM to know how to search *our* data

In this post, I’ll walk through each. With a bit more breadth, you might appreciate which flavor your colleagues seem to mean when they say “agentic search”

Retrieval-Centric Implementation

When frontier models don’t know, they search. Ask ChatGPT a question about news, it’ll search. Ask it about a very specific technical problem, it’ll search. During training, LLMs see *search* examples as a technique to learn what it doesn’t know. To find what it needs.

Therefore, we just need to build good search to solve any conceivable query from an agent.

Let’s assume we run an e-commerce catalog. For us, when users search “red shoes” they mean “red high heels”. But the agent doesn’t know that. Luckily it asks *search*. Below we see it returns the right results.

[Daily article] July 12: Wikipedia:Featured topics/Liverpool F.C.

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 12 JUL 2026 · [SOURCE](#)

Since 1985, Liverpool Football Club has seen the appointment of Kenny Dalglish as manager, the Hillsborough disaster, and the club's return to European competition. Liverpool won a Premier League and FA Cup double in Dalglish's first season, and won further league titles in 1987–88 and 1989–90. A crush of people at Hillsborough Stadium in April 1989 resulted in 96 deaths (memorial pictured) and led to the end of standing terraces. In 2001, Liverpool won a unique treble of trophies, and a year later finished second to Arsenal, their highest league finish in 11 years. Liverpool won the UEFA Champions League for the fifth time in Rafael Benítez's debut season in 2005, and he guided the club to another FA Cup victory in 2006. Since 2015, Jürgen Klopp has led Liverpool to consecutive Champions League finals; the 2019 win sealed a sixth Champions League title and ended a seven-year trophy drought, followed by a win in the 2019 UEFA Super Cup. (

Read more: https://en.wikipedia.org/wiki/Wikipedia:Featured_topics/Liverpool_F.C.

Today's selected anniversaries:

1963:

Sixteen-year-old Pauline Reade, the first victim of serial killers Ian Brady and Myra Hindley, disappeared in Gorton, England. https://en.wikipedia.org/wiki/Moors_murders

1979:

Rowdy fans at Comiskey Park in Chicago stormed the field during a Major League Baseball promotional event at which a crate of disco records was blown up. https://en.wikipedia.org/wiki/Disco_Demolition_Night

1998:

France defeated the defending champions Brazil 3–0 to win their first FIFA World Cup title. https://en.wikipedia.org/wiki/1998_FIFA_World_Cup_final

2006:

Why I think human consciousness isn't comparable to an LLM

The following is uncomfortably philosophical, but: In my worldview, humans are dramatically different things than a function . For hundreds of millions of years, nature generated new versions, and only a small number of these versions survived. Human thought is a poorly-understood process, involving enormously many neurons, extremely high-bandwidth input, an extremely complicated cocktail of hormones, constant monitoring of energy levels, and millions of years of harsh selection pressure.

We understand essentially nothing about it. In contrast to an LLM, given a human and a sequence of words, I cannot begin putting a probability on "will this human generate this sequence".

To repeat myself: To me, considering that any human concept such as ethics, will to survive, or fear, apply to an LLM appears similarly strange as if we were discussing the feelings of a numerical meteorology simulation.

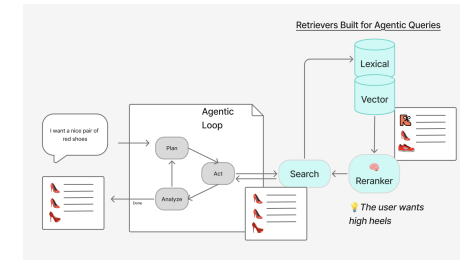
The real issues

The function class represented by modern LLMs are very useful. Even if we never get anywhere close to AGI and just deploy the current state of technology everywhere where it might be useful, we will get a dramatically different world. LLMs might end up being similarly impactful as electrification.

My grandfather lived from 1904 to 1981, a period which encompassed moving from gas lamps to electric, the replacement of horse carriages by cars, nuclear power, transistors, all the way to computers. It also spanned two world wars, the rise of Communism and Stalinism, almost the entire lifetime of the USSR and GDR etc. The world on his birth looked nothing like the world when he died.

Navigating the dramatic changes of the next few decades while trying to avoid world wars and murderous ideologies is difficult enough without muddying our thinking.

Below we see initial retrieval to lexical / vector backends pulls back some reasonable, if naive, “red shoe” candidates. Still that’s not quite right. Luckily the reranker shapes the results towards our understanding of that intent.



Of course we might have other components here - query understanding, diversity, custom embedding models, and more.

The important point is that *search leads the agent by the nose* towards what’s relevant. We assume search can define what a good “red shoe” is, overriding the agent’s perspective.

Most teams build retrieval-centric approaches with classic RAG. Chunks of answers, and embeddings trained to recognize them as answers.

Unfortunately, answers don’t always look tied to the question. For example:

Question:

Synopsis of the book Ubik

Answer:

By the year 1992, humanity has colonized the Moon and psychic powers are common. The protagonist, Joe Chip, is a debt-ridden technician working for Runciter Associates, a “prudence organization” employing “inertials”—people with the ability to negate the powers of telepaths and “precogs”—to enforce the privacy of clients.

If you don’t know the book Ubik, it’s not clear that this answers the question. The agent says “cool story bro” and ignores the info.

Search like this actually is divorced from the web search trusted by frontier models. The web contains titles, headings, and other elements placing the answer in context.

The big downside? Search remains hard. We're not Google.

Most importantly, we don't have perfect search like Google or Bing. Even with good search - almost nobody builds *google quality results*. And since agents trust search - simple distractors in retrieval can, as Lester Solbakken says, easily confuse reasoning.

Harness-centric implementations

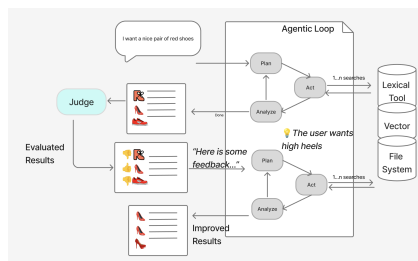
Who should be in charge? Should the agent manage the search process? Or should search drive the agent?

Moving the emphasis to the harness, we put the agent in charge.

Imagine stripping search tools down to core retrieval primitives. Just a BM25 backend. Or a filesystem with CLI tools. We tell the agent find to what it needs with these untuned tools. The agent might struggle more, but hey, its smart, it can get figure it out. Right?

Still, the agent might find what it thinks is relevant, as it does in the image below. Sadly that's not what's actually relevant. To help the agent, we inject external knowledge. We let a judge direct the agent, correcting its mistakes, and guiding it towards better search strategies.

So when the agent returns results, it's not the "user" that receives the candidates. Instead a judge labels results as relevant or not. The agent gets the hint, finding results similar to those labeled relevant. Avoiding those labeled irrelevant. To steel from Jo Kristian Bergem, this is relevance feedback on steroids.



This works. Look at the ESCI dataset. If I have an oracle labeling with judgments from these datasets, I get quite significant improvements.

Variant	NDCG@10	Description
ESCI BM25	0.2895	Simple BM25 weighing name / description
ESCI agentic	0.4101	GPT5-mini tool calling loop w/ BM25+e5 embeddings tools

I am baffled that the AI discussions seem to never move away from treating a function to generate sequences of words as something that resembles a human. Statements such as "an AI agent could become an insider threat so it needs monitoring" are simultaneously unsurprising (you have a randomized sequence generator fed into your shell, literally **anything** can happen!) and baffling (you talk as if you believe the dice you play with had a mind of their own and could decide to conspire against you).

Instead of saying "we cannot ensure that no harmful sequences will be generated by our function, partially because we don't know how to specify and enumerate harmful sequences", we talk about "behaviors", "ethical constraints", and "harmful actions in pursuit of their goals". All of these are anthropocentric concepts that - in my mind - do not apply to functions or other mathematical objects. And using them muddles the discussion, and our thinking about what we're doing when we create, analyze, deploy and monitor LLMs.

This muddles the public discussion. We have many historical examples of humanity ascribing bad random events to "the wrath of god(s)" (earthquakes, famines, etc.), "evil spirits" and so forth. The fact that intelligent highly educated researchers talk about these mathematical objects in anthropomorphic terms makes the technology seem mysterious, scary, and magical.

We should think in terms of "this is a function to generate sequences" and "by providing prefixes we can steer the sequence generation around in the space of words and change the probabilities for output sequences". And for every possible undesirable output sequence of a length smaller than , we can pick a context that maximizes the probability of this undesirable output sequence.

A much clearer formulation, which helps more clearly articulate the problems to solve.

Why many AI luminaries tend to anthropomorphize

Perhaps I am fighting windmills, or rather a self-selection bias: A fair number of current AI luminaries have self-selected by their belief that they might be the ones getting to AGI - "creating a god" so to speak, the creation of something like life, as good as or better than humans. You are more likely to choose this career path if you believe that it is feasible, and that current approaches might get you there. Possibly I am asking people to "please let go of the belief that you based your life around" when I am asking for an end to anthropomorphization of LLMs, which won't fly.

"Alignment" for LLMs

Alignment and safety for LLMs mean that **we should be able to quantify and bound the probability with which certain undesirable sequences are generated**. The trouble is that we largely fail at describing "undesirable" except by example, which makes calculating bounds difficult.

For a given LLM (without random seed) and sequence, it is trivial to calculate the probability of the sequence to be generated. So if we had a way of somehow summing or integrating over these probabilities, we could say with certainty "this model will generate an undesirable sequence once every N model evaluations". We can't, currently, and that sucks, but at the heart, this is the mathematical and computational problem we'd need to solve.

The surprising utility of LLMs

LLMs solve a large number of problems that could previously not be solved algorithmically. NLP (as the field was a few years ago) has largely been solved.

I can write a request in plain English to summarize a document for me and put some key datapoints from the document in a structured JSON format, and modern models will just do that. I can ask a model to generate a children's book story involving raceboats and generate illustrations, and the model will generate something that is passable. And much more, all of which would have seemed like absolute science fiction 5-6 years ago.

We're on a pretty steep improvement curve, so I expect the number of currently-intractable problems that these models can solve to keep increasing for a while.

Where anthropomorphization loses me

The moment that people ascribe properties such as "consciousness" or "ethics" or "values" or "morals" to these learnt mappings is where I tend to get lost. We are speaking about a big recurrence equation that produces a new word, and that stops producing words if we don't crank the shaft.

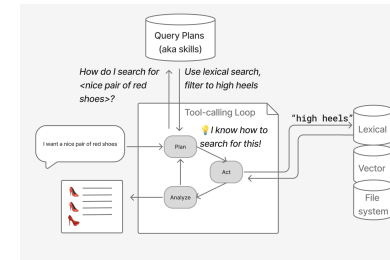
To me, wondering if this contraption will "wake up" is similarly bewildering as if I was to ask a computational meteorologist if he isn't afraid of his meteorological numerical calculation will "wake up".

ESCI agentic w/
oracle 0.5843

GPT5-mini tool calling loop w/ BM25+e5 embeddings tools.
Judge responds with feedback once

Here the second row we trust the agent's interpretation of queries. That provides a large gain over BM25. But inject more domain knowledge via a judge (the third row) and agent performance improves dramatically.

We can guide agents with information ahead of time too. For example, I've seen teams take inspiration from skills in coding agents. Imagine a targeted query plan that tells the agent how to use it's tools to solve a specific problem.



Optimizing content helps harnesses

We help agents even further by optimizing content for findability. That's the trick we've played on ourselves with coding agents. We write docs to explain to agents why something's useful. We don't just rely on that dumb chunk, we document the purpose of the knowledge:

```
/books/scifi/ubik.txt
```

```
## About Book Ubik Ubik is a book by Philip K. Dick
```

```
## Synopsis
```

```
By the year 1992, humanity has colonized the Moon and psychic powers are common. The protagonist, Joe Chip, is a debt-ridden technician working for Runciter Associates, a "prudence organization" employing "inertials"—people with the ability to negate the powers of telepaths and "precogs"—to enforce the privacy of clients.
```

Unlike a naive chunk, this bit of information has purpose. It's clear, when its in the context, what problem it solves.

By organizing + cataloging information we've done what every search developer wishes their content team would do: actually optimize content to be findable.

The downside: cost

The major downside here? Token costs. The exploration here requires more than just issuing a search and retrieving one set of results. It's an agent's exploration of the environment. And that exploration starts fresh with every new context window.

Model-centric approach

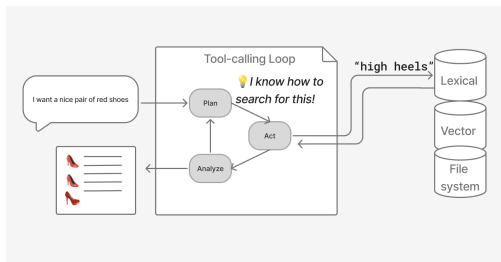
What if we literally fine-tuned the LLM to search efficiently?

The harness based approach takes a long, winding path towards relevant results. With every fresh context, it wastes calls relearning what works over-and-over. It's costly and slow.

Could we train a model that doesn't repeat the same mistakes with our tools?

Our judge helps label some of the agent's reasoning / tool-calling paths as useful. And label others less so. Once you have labeled traces, you can fine-tune an open weights model to proactively, efficiently make the right tool calls given the input.

Below, for example, the model knows exactly how to search for our "red shoes" query given its extensive fine tuning.



Now we've built a model targeting the search task. We don't tune our retrievers much. Instead we continue to use the simpler / dumb search tools.

Even cooler, the agentic search model's behavior can change just by adjusting the system prompt. "This is an e-commerce search task, optimize for top 10 ranking" might be different from "You're retrieving chunks for financial research, maximize diversity and relevance at the same time."

Further the models can stay smaller. By just focusing on the search part - not the entire domain problem - they can shrink to a manageable size allowing them to be self hosted.

The space of words

The tokenization and embedding step maps individual words (or tokens) to some vectors. So let us imagine for a second that we have in front of us. A piece of text is then a path through this space - going from word to word to word, tracing a (possibly convoluted) line.

Imagine now that you label each of the "words" that form the path with a number: The last word with 1, counting forward until you hit the first word or the maximum context length. If you've ever played the game "Snake", picture something similar, but played in very high-dimensional space - you're moving forward through space with the tail getting truncated off.

The LLM takes your previous path into account, calculates probabilities for the next point to go to, and then makes a random pick into the next point according to these probabilities. An LLM instantiated with a fixed random seed is a mapping of the form .

In my mind, the paths generated by these mappings look a lot like strange attractors in dynamical systems - complicated, convoluted paths that are structured-ish.

Learning the mapping

We obtain this mapping by training it to mimic human text. For this, we use approximately all human writing we can obtain, plus corpora written by human experts on a particular topic, plus some automatically generated pieces of text in domains where we can automatically generate and validate them.

Paths to avoid

There are certain language sequences we wish to avoid - because the sequences these models generate try to mimic human speech in all its empirical structure, but we feel that some of the things that humans have empirically written are very undesirable to be generated. We also feel that a variety of other paths should ideally not be generated, if - when interpreted by either humans or other computer systems - undesirable results arise.

We can't specify strictly in a mathematical sense which paths we would prefer not to generate, but we can provide examples and counterexamples, and we try to hence nudge the complicated learnt distribution away from them.

That's much more my interest these days. When someone says "LLM as a judge" I nod and say yes let's build that, but then go on and probably look into using LLM features heavily in a larger model. LLMs can be very useful for rapid feature development: they're promptable and easy to play with.

If you're interested in this approach, check out this [blog post](#) and the corresponding [Haystack talk](#). Also a talk from Shopify [choosing cross encoders for evaluation over LLMs](#)

Enjoy!

Enjoy softwareDoug in training course form!



I hope you join me at [Cheat at Search with LLMs](#) to learn how to apply LLMs to search applications. Check out [this post](#) for a sneak preview.

A non-anthropomorphized view of LLMs

HALVAR.FLAKE · 14 SEP 2025 · [SOURCE](#)

In many discussions where questions of "alignment" or "AI safety" crop up, I am baffled by seriously intelligent people imbuing almost magical human-like powers to something that - in my mind - is just MatMul with interspersed nonlinearities.

In one of these discussions, somebody correctly called me out on the simplistic nature of this argument - "a brain is just some proteins and currents". I felt like I should explain my argument a bit more, because it feels less simplistic to me:

The downside? Early days

The OG in the space is [SID](#). Other contenders like Glean's Waldo model have sense entered. They're all promising, but not widely adopted. Many unknown unknowns exist. In a sense, we don't yet know the upside or downsides yet. I suspect though this paradigm has tremendous promise in the years ahead.

In other words - stay tuned!

The alchemy of ambiguity

People use "agentic search" confusingly and inconsistently. They focus on their part of the problem. That means we have a collection of solutions like enriched base metals - and its unclear what alchemy brings them together.

We're converging on a fascinating mix of easier to organize data (like [PageIndex](#)). I'm seeing patterns in harness design that mirror coding agents - hooks / evaluators that steer agents. Or "skills" that guide them a priori. Much like with coding - the agents eat the harnesses. When a successful pattern emerges, agents, in our case agentic search models, train to memorize these patterns.

Retrieval also seems to be moving towards agent-centric approaches. Agents have [their own specific, unique retrieval patterns](#). We're finally seeing late interaction + learned sparse retrieval approaches have their day from companies like [LightOn](#) and [MixedBread](#)

We're swimming in an interesting retrieval primordial goop, what comes out may be an obvious combination - or look nothing like the component parts. Exciting times!

Enjoy softwareDoug in training course form!

Starting May 18!

Signup here - <http://maven.com/softwareDoug/cheat-at-search>



I hope you join me at [Cheat at Search with Agents](#) to learn use agents in search, build better RAG and use LLMs in query understanding.

LLM Judges aren't the shortcut you think

DOUG TURNBULL · 03 NOV 2025 · [SOURCE](#)

Is “LLM as a judge” overhyped? After years of implementing LLM judges for clients, I find more of them recognize the limitations.

LLM judges fail for the same reason any evaluation method fails: teams lack good data+evaluation to begin with. LLM judges aren't the shortcut teams hope. Instead, they are themselves a thing that needs data and monitoring to get use out of.

Let me walk through the pitfalls of LLM as a judge (for search) that you'll encounter.

LLM as a judge

When I talk about LLM as a judge, my focus is search. I mean creating a [judgment list](#). For search that means labeling how relevant a document is for a query. How relevant is the product “garden trowel” for a `q=shovels`? Maybe this “garden trowel” is less relevant than “iron spade”?

In the ancient past, to improve the search, I would be limited to a tiny hand-labeled judgment list. Labeling by hand is expensive. Datasets drift out of date quickly. New use cases arise. So we constantly need humans to relabel.

How to LLM as a judge

Here's how I use LLM judges:

- To extend labels on use cases I already measure decently to new queries (because I can trust alignment of the LLM on this use case)
- As a safeguard against regressions

I trust the LLM judge to flag a regression on the use cases my human labels cover. The LLM is aligned and measured against these. I don't trust the LLM to discover new, unexpected ways users search. I look to the users for that, not LLMs.

Still many teams don't even do decent monitoring of topical relevance. There are plenty of obviously broken search engines. The ones that would benefit from an LLM judge, likely would benefit from rounds of human labeling first needed before blindly trusting an LLM judge.

We already have automated data labeling at home kids

If we have enough labels to evaluate LLM as a judge on a use case, we have enough to train an ML model to recover those labels. We can then see a better place LLM's fit into evaluation:

- LLMs focus on factual, topical parts of the problem. But miss other parts of the problem
- LLMs might see a subset of the content features
- Other non-LLM features might help us better evaluate relevance (embedding similarity, text matches, etc)
- Other ground-truths (besides human) might be a better definition of relevance (ie engagement based labels)

Instead of LLM as a Judge, **why not Learning to Rank as a judge?** We seem to have forgotten that ranking models provide value beyond directly deploying them to production. They can help guide more manual search tuning as well.

I'm becoming more of a fan of LLM-for-feature-generation over LLM as a judge. LLMs can cover the topical part of relevance. You can ask them “Which of these product names is more relevant to the query?”. The downstream, traditional ML model is agnostic to where the features come from. They could be an embedding similarity, LLM decision, lexical similarity, or more.

- How comprehensively you’re describing products to an LLM?

Really it gets down to a classic problem in search. There’s perceived relevance - the relevance of the product, as it appears in your product page. Then there’s actual relevance - the detailed, thoughtful understanding of your product by a user. A lot depends not just on true relevance, but how it’s presented to users.

Problem five - Your LLM judge can’t be trusted on novel use cases

I assume the goal of LLM as a judge in to align to human labelers. But rarely do those judgments comprehensively represent every use case.

Search metrics with human evals don’t go up-and-to-the-right. They seesaw as teams reactively label, solve, label, solve ad infinitum. In my experience it’s rare to find a team build a true, comprehensive human labeled evals covering every use case. Even if we did, we’d need to account for evolving search use cases.

The flow usually goes like this: we want to solve a problem (search by product name). So we capture some judgments for that use case. Relevance starts low. We work a bit to improve this use case and it goes up. Yay!

But the boss comes to our desk and suggests we have a new problem to solve: search of furniture by the room (living room, etc). We gather more labels. We know we’re bad at this use case. So search metrics go back down. Sad 😞

We solve that use case, it goes back up to where it was before. Repeat. Up and down. See-saw.

This presents a problem for an LLM judge. If our human labels only focus on a subset of use cases, **we’re also aligning our LLM judges on a subset of use cases.**

Our LLM judge generalizes to just a piece of the problem. The LLM judge can’t be trusted on a new use case until we measure it’s abilities in that use case.

So if you seesaw its human labeling, you must seesaw the alignment of the LLM judge to the use cases we capture in human eval. As we adopt a new use case, we need more human labels. We need to retest the LLM judge. And so on.

The task of human labeling does not end with LLM Judges. In fact its even more crucial.

These costs inspire teams to use LLMs to build judgments, They:

1. Start with a ground truth of human labels
2. Build a prompt to approximate that ground truth
3. Apply those prompts to go beyond the ground truth to new query / documents

If done well, we can sit at our laptop, tinker with relevance algorithms, and continue to tune without bother humans for non-stop relabeling.

Sounds promising. But teams often ignore many of the assumptions and nuance underlying the task of LLM labeling. Let’s go through them, and we’ll see at the end, maybe LLMs should play a different role in evaluation?

Problem one - LLM evals don’t know what’s engaging to users

Users’ ever-evolving lizard brains drive conversions. And while optimizing for just lizard brains would itself be a nightmare, so would only focusing on an LLM’s opinion.

LLMs do a better job approximating *human evaluations*. In search, we call these evaluations *explicit* judgments. They use human labelers’ basic knowledge of the world to label a result as *topically relevant* to the query or not.

Topical relevance is only one kind of relevance. As an example, for a search of “articles about harry potter”. The article is either factually about Harry Potter or its not. We might also consider other gradations:

- Is the article well written?
- Is it article from an authoritative source?
- Is the article about a tangential topic? Dumbledore vs Gandalf etc?

All of which relate to topicality, authority, and knowledge. An LLM excels at this — LLMs live in a world of facts and knowledge.

But LLMs don’t have limbic systems. So their evaluations come with limitations.

In real search applications, “relevance” goes beyond topicality. I worked at Reddit on search. A search for “harry potter” might mean anything from a memes, to drama about JK Rowling, to every kind of juicy controversy, spicy post, or oddity. People search for “cybertruck” not for reviews but to see silly videos.

Then there’s the more mundane. I found at Shopify, users preferred plain or black clothing over flashy items based on their clicks. Unless you tell an LLM this (because you’ve done the analysis) an LLM won’t intrinsically catch this.

Often the “topical” part of search is relatively easy. But engaging search, that captures these human oddities, must go beyond world-knowledge to at least get *some* of a user’s lizard brain.

Problem two - the last 10% of disagreement is the important stuff

Many teams get excited when they immediately see 70-80% agreement with human labelers. With a bit of tuning, they can inch closer to 90%.

In my experience, that last bit of human-LLM disagreement matters. Those are the non-obvious cases. Search deals with a big hard negatives challenge. Those examples that deceptively seem relevant naive search algorithms (or labelers), but are, in fact irrelevant. That last 10-30% might be what more careful labelers would catch. Or represented in clickstream based labels.

For example on this furniture dataset, a LLM doesn’t understand that a “bistro table” is meant for **outdoor** use in US furniture lingo. That’s not obvious to an LLM: The LLM imagines I’m opening a restaurant.

Simple algorithms often capture that first 80-90%. It’s the 10-20% you need great labels for. So pay attention not just to coverage. But for which use cases.

A 90% coverage might be good regression tests, it might not help you go beyond easier wins.

Problem three - sneaky overfitting

So far, I’ve mentioned a few strange cases the LLM judge might miss: a bistro table is meant for outdoor usage. Users preferring darker clothing items.

The savvy prompt engineer will want to fill the prompt with examples to help the LLM perform better on these cases.

But beware overfitting to these particular failure cases.

With every new rule, you push the model’s meager attention away from the general problem, and towards your exceptions. Layer example after example, and you’ll find yourself with a brittle system focused excessively on specific use cases.

Overfitting with prompts can be sneaky. Playing whack-a-mole with your eval data can degrade generality. You can get perfect results on the eval set, only to expand to unseen queries, and find the gains no longer hold.

To do a good job, you need holdout data you’re not tuning against. A set of judgments you don’t peak at. Because peaking might cause you to sneak a change into the prompt, making the holdout less useful as an independent validation step.

Add rules and exceptions carefully, you’ll be surprised how easy it is to pull an LLM away from its generality.

Problem four - LLMs only evaluate the document it sees

LLMs only evaluate the document it sees. IE:

```
Query: outdoor seating
Product Name: Bistro Table
Product Description: Enjoy this lovely bistro table on your porch...
```

Is this enough information for evaluating the product? Is it what a user sees?

Suddenly, you decide to add new information about the product

```
Query: outdoor seating
Product Name: Bistro Table
Product Description: Enjoy this lovely bistro table on your porch...
Category: Outdoors
```

What are we evaluating truly?

- The products relevance?
- The usefulness of a set of product properties for determining relevance?